

**An unjustified answer will be considered as false**

**It is imperative to use the provided squared sheets for the simplified diagrams**

Consider the P9 pipeline processor. This processor has the same instruction set as the **Mips**. It is built around a 9-stage pipeline : **IF1**, **IF2**, **DEC1**, **DEC2**, **EXE1**, **EXE2**, **MEM1**, **MEM2**, **WBK**

This pipeline has been divided into stages referring to the Mips processor pipeline. Each of the **IFC**, **DEC**, **EXE** and **MEM** stages has been divided into two stages.

**However, the calculation of the next instruction address is achieved in the EXE2 stage.**

The implementation contains all the bypasses required to support data dependencies. However, there are no bypasses arriving to the **MEM1** stage.

We wish to study the performance of execution of a distance calculation function on this implementation.

Consider the **Dist** function. This function takes as input an array of integers **x** and calculates the value of the elements of an array **y** such that :  $y(i) = |x(i) - x(i + 1)|$

```
void Dist (int *x, int *y, int size)
{
    int i;
    for (i=0 ; i!=(size-1) ; i++)
    {
        if (x[i] < x[i+1])
            y [i] = x [i+1] - x [i ];
        else
            y [i] = x [i ] - x [i+1];
    }
}
```

We have obtained two different versions of the **Dist** main loop assembly code from a non-pipelined Mips assembler.

At the loop entry, the register R4 is assumed to contain the address of array **x**, R5 the address of array **y**, and R6 the address of the last element of array **x**.

**Version 1 :**

```
Loop:  Lw      r8 , 0 (r4 )      ; x[i ]
       Lw      r9 , 4 (r4 )      ; x[i+1]
       Sub     r10, r8, r9
       Bgez    r10, _Endif
       Sub     r10, r9 , r8

_Endif: Sw     r10, 0 (r5 )      ; y[i] = abs(x[i]-x[i+1])
       Addu    r5 , r5 , 4
       Addu    r4 , r4 , 4
       Bne     r4 , r6 , Loop
```

### Version 2 :

```
Loop: Lw    r8 , 0 (r4 )      ; x[i ]
      Lw    r9 , 4 (r4 )      ; x[i+1]
      Sub   r10, r8, r9

      Slt   r11, r10, r0
      Sub   r12, r0 , r11
      Xor   r10, r10, r12
      Add   r10, r10, r11

      Sw    r10, 0 (r5 )      ; y[i] = abs(x[i]-x[i+1])

      Addu  r5 , r5 , 4
      Addu  r4 , r4 , 4
      Bne   r4 , r6 , Loop
```

Throughout the exercise, we assume that in the array  $\mathbf{x}$ , the probability that :  $x(i) < x(i + 1)$  is 40%.

- 1- Does the 9-stage pipeline have any impact on the compiler ?
- 2- In the Mips processor, we have data dependencies of order 1, 2 and 3. What are the data dependencies in P9 ?
- 3- What bypasses are required to execute instructions in this P9 ? Illustrate the use of each bypass with an example of instruction sequence.
- 4- For Version 1, modify the loop code so that it becomes executable on the P9 realization. Analyze the execution of this loop using a simplified diagram, assuming that the branch fails. Calculate the average number of cycles per iteration. Calculate the CPI and CPI-useful.
- 5- For Version 2, modify the loop code so that it becomes executable on the P9 realization. Analyze the execution of this loop using a simplified diagram. Calculate the average number of cycles per iteration. Calculate the CPI and CPI-useful.
- 6- For each of the two versions, propose an optimized code by modifying the order of the instructions. You don't need to draw a diagram, but simply indicate the freeze cycles on the optimized code. For each of the two versions, calculate the average number of cycles per iteration. Calculate the CPI and CPI-useful.
- 7- For each of the two versions, unroll the loop code once (processing 2 elements at each iteration) and optimize. You don't need to draw a diagram, but simply indicate the freeze cycles on the code after optimization. For each of the two versions, calculate the average number of cycles per iteration, the CPI and the CPI-useful.
- 8- For each of the two versions, optimize the loop code using the *software pipeline* technique. Describe clearly your approach. Specify the number of stages and the instructions included in each stage. You don't need to draw a diagram, but simply indicate the freeze cycles on the code after optimization. For each of the two versions, calculate the average number of cycles per iteration, the CPI and the CPI-useful.